

Preparation Report – LAB 3

ADVANCED CPU ARCHITECTURE AND HARDWARE ACCELERATORS LAB

Name: Amit Postelnik ,Yuval Elron

ID: 206458333, 318676061

Introduction

In this laboratory we implemented a simple multi-cycle CPU architecture that includes a Control Unit and a Datapath unit. The processor executes instructions according to the given ISA and supports arithmetic, logical, memory and branch operations.

The Control Unit was implemented using a Mealy FSM, while the Datapath executes the required operations according to the control signals generated by the FSM.

The system receives two initialization text files. The first file is ITCMinit.txt, which initializes the instruction memory. The second file is DTCMinit.txt, which initializes the data memory.

At the end of execution, the processor updates the Data Memory content according to the executed program.

System Architecture

The processor contains the following main modules: Control Unit, Datapath, Program Memory (ITCM), and Data Memory (DTCM).

The Control Unit decodes the current instruction and generates the required control signals. The Datapath performs arithmetic and logical operations, memory accesses and register operations. The Program Memory stores the instructions, while the Data Memory stores the program data.

3. Controller-based system:

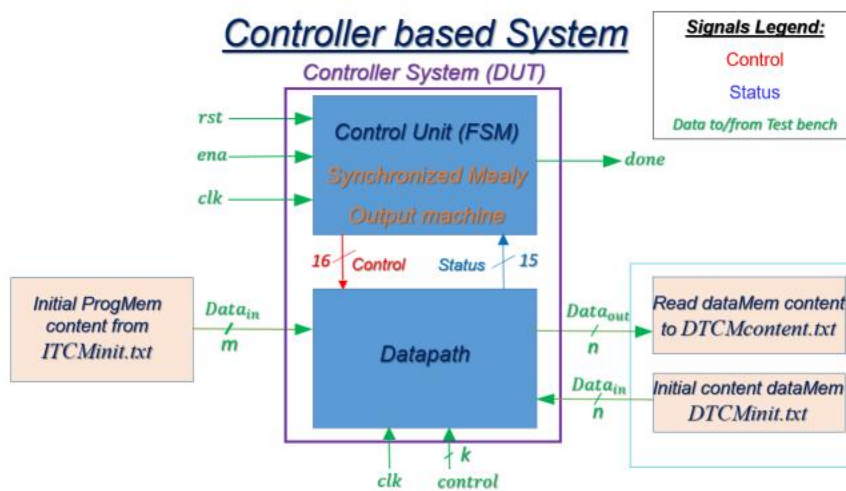


Figure 1: Overall DUT structure

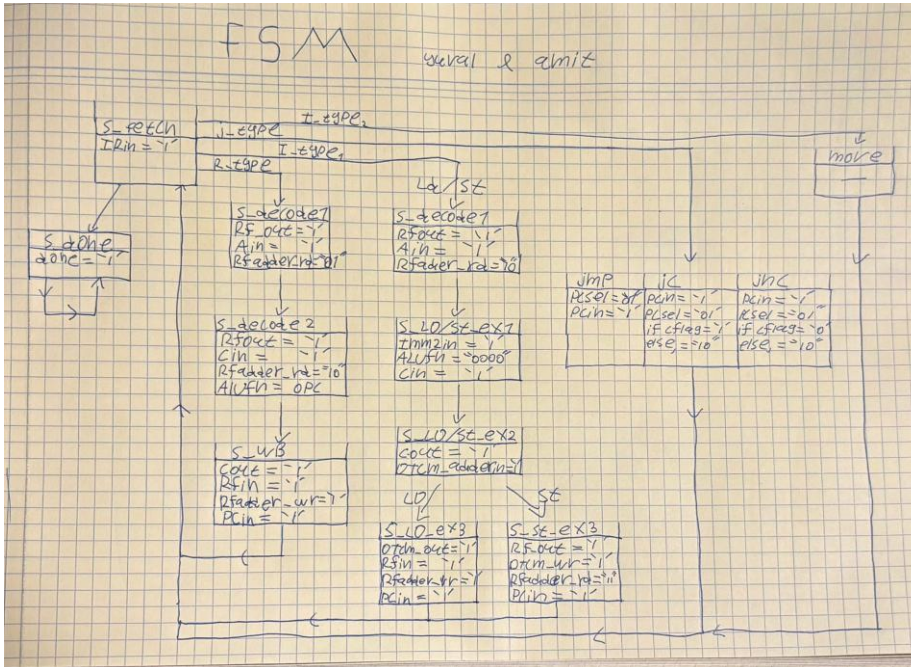
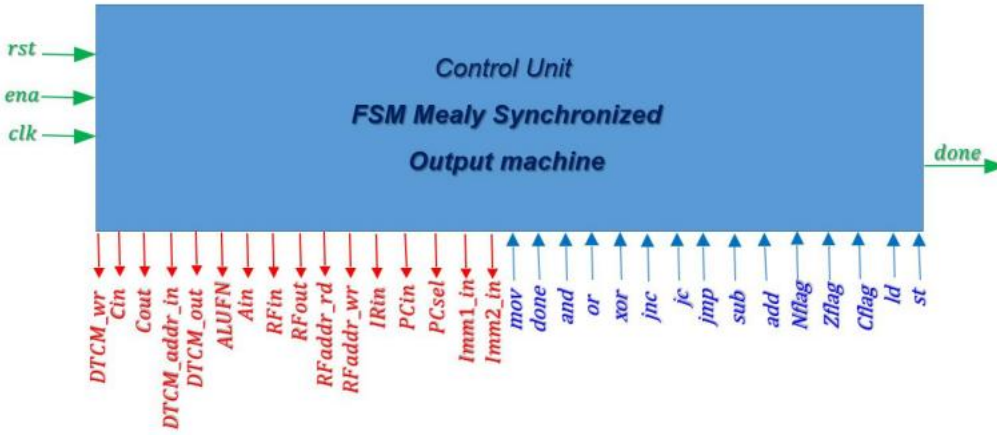
Control Unit

The Control Unit is responsible for controlling the processor operation. The unit receives status signals from the Datapath and generates the required control signals according to the current FSM state and opcode.

The FSM includes several stages such as Fetch, Decode, Execute, Memory Access and Write Back. Different instruction types require different numbers of clock cycles.

Control Unit

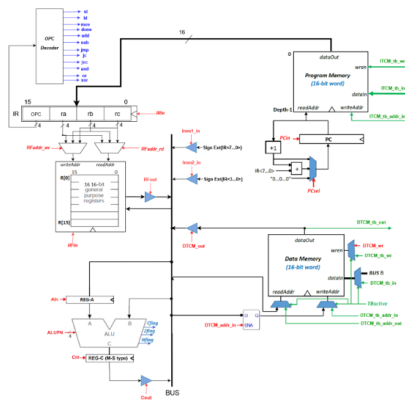
Signals Legend: Control, Status, Data to/from Test bench



Datapath Unit

The Datapath unit performs all arithmetic and logical operations in the processor. The Datapath contains the Register File, ALU, A/B/C registers, Program Counter, internal BUS and memory interfaces.

The ALU performs operations such as ADD, SUB, AND and MOV. The Datapath receives control signals from the Control Unit and updates the internal registers and memories accordingly.



Program Description

The implemented assembly program corresponds to Program 5. The program works on two arrays stored in the Data Memory: arr1 and arr2. Each array contains 14 values, and the result of the program is stored in a third array called res.

The program scans both arrays element by element using an index i. For each index, the program checks whether i is odd or even.

If i is odd, the program adds the matching values from arr1 and arr2 and stores the result in res[i].

$$\text{res}[i] = \text{arr1}[i] + \text{arr2}[i]$$

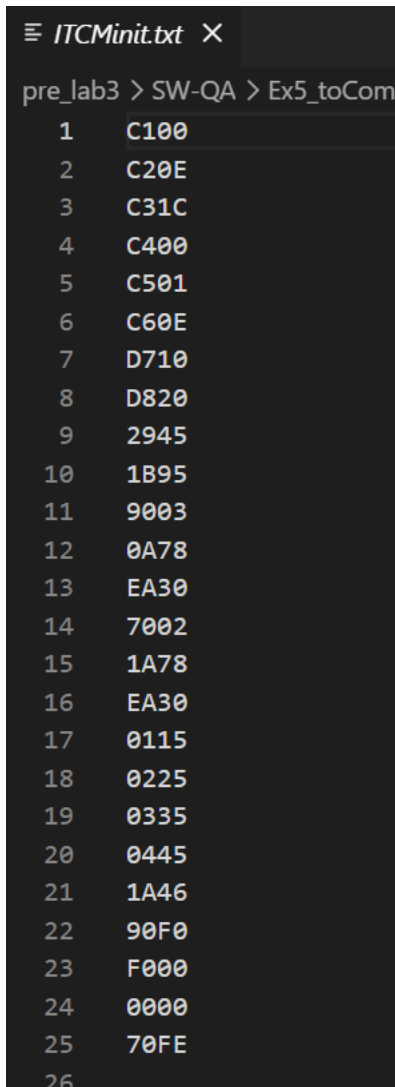
If i is even, the program subtracts the matching values and stores the result in res[i].

`res[i] = arr1[i] - arr2[i]`

At the end of execution, the res array contains the result of all 14 operations.

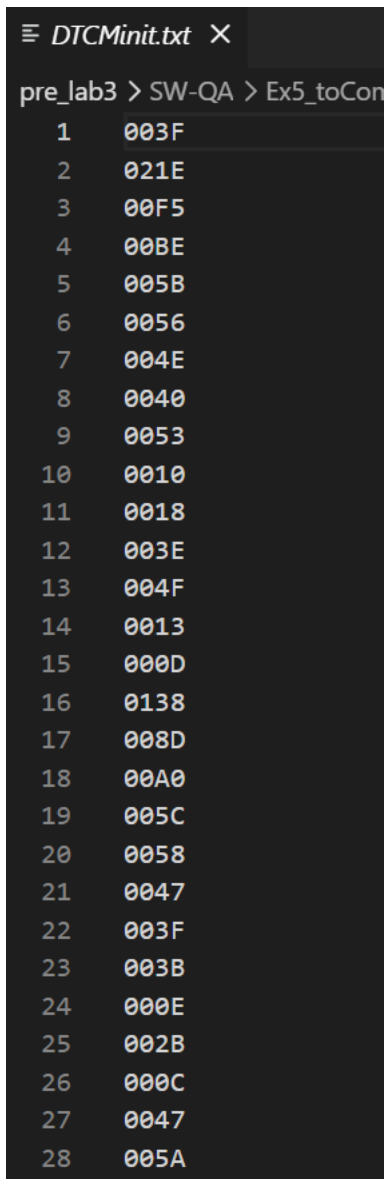
ITCM and DTCM Initialization

The ITCM file contains the encoded instructions of the program. These instructions implement the loop, the odd/even index check, the addition operation, the subtraction operation, and the store operation into the result array.



```
ITCMinit.txt X
pre_lab3 > SW-QA > Ex5_toCom
1 C100
2 C20E
3 C31C
4 C400
5 C501
6 C60E
7 D710
8 D820
9 2945
10 1B95
11 9003
12 0A78
13 EA30
14 7002
15 1A78
16 EA30
17 0115
18 0225
19 0335
20 0445
21 1A46
22 90F0
23 F000
24 0000
25 70FE
26
```

The DTCM file contains the initial values of arr1 and arr2, followed by empty memory locations reserved for the res array.

A screenshot of a terminal window with a dark background. The title bar at the top shows a hamburger menu icon, the filename 'DTCMinit.txt', and a close button 'X'. The terminal prompt is 'pre_lab3 > SW-QA > Ex5_toCon'. Below the prompt, there is a list of 28 memory addresses and their corresponding hexadecimal values, numbered 1 through 28 on the left.

1	003F
2	021E
3	00F5
4	00BE
5	005B
6	0056
7	004E
8	0040
9	0053
10	0010
11	0018
12	003E
13	004F
14	0013
15	000D
16	0138
17	008D
18	00A0
19	005C
20	0058
21	0047
22	003F
23	003B
24	000E
25	002B
26	000C
27	0047
28	005A

Simulation Results

The simulation verifies the correct execution of the processor. The waveform shows instruction fetch operations, ALU calculations, register updates, memory accesses and branch operations.

The processor successfully scans both arrays, checks the parity of the index, performs addition for odd indexes and subtraction for even indexes, and stores each result in the res array.

Final Memory Content

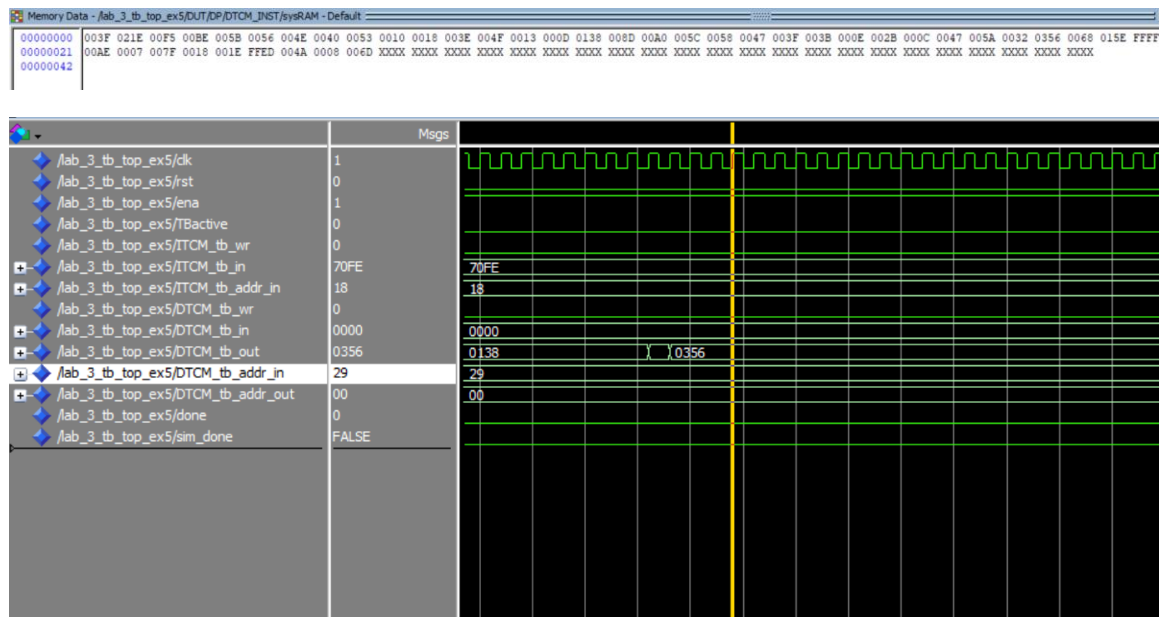
At the end of the program, the expected result array is:

res = {50, 854, 104, 350, -1, 174, 7, 127, 24, 30, -19, 74, 8, 109}

Since the processor uses 16-bit values, negative results are represented in two's complement.

The expected memory values in hexadecimal are:

29	0032
30	0356
31	0068
32	015E
33	FFFF
34	00AE
35	0007
36	007F
37	0018
38	001E
39	FFED
40	004A
41	0008
42	006D



Conclusion

The processor successfully executed Program 5 according to the required ISA. The Control Unit FSM, Datapath and memory modules operated correctly. The

simulation results matched the expected behavior, and the final Data Memory content contains the correct values in the res array.